

Doc 8.2 — `switch`, Ternary & Conditional Expressions

এই Doc-এ কী শিখব?

JavaScript-এ একই ধরনের decision বিভিন্নভাবে লেখা যায়।

যেমন:

- অনেকগুলো fixed value check করা → `switch`
- ছোট দুই-way decision → ternary `? :`
- React JSX-এর মধ্যে conditional UI → ternary
- Complex logic → `if / else`

এই Doc-এ আমরা শিখব:

1. `switch` কী
2. `case`
3. `break`
4. `default`
5. Fall-through
6. Multiple cases
7. `switch` comparison behavior
8. `switch` with strings
9. `switch` with numbers
10. `switch` vs `if / else`
11. Ternary operator
12. Ternary syntax
13. Ternary as an expression
14. Ternary with variables
15. Ternary with function return
16. Ternary in React JSX
17. Nested ternary
18. Ternary vs `if / else`
19. Readability & best practices
20. Common mistakes
21. Practice questions
22. Coding challenges
23. Interview questions
24. Quick revision

1. `switch` কী?

`switch` হলো JavaScript-এর একটি **conditional control-flow statement**।
একটি value-এর সাথে অনেকগুলো fixed value compare করার জন্য `switch` useful।

Example:

```
const day = "Monday";

switch (day) {
  case "Saturday":
    console.log("Weekend");
    break;

  case "Sunday":
    console.log("Weekend");
    break;

  case "Monday":
    console.log("Working day");
    break;

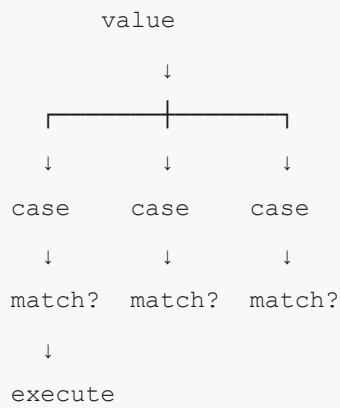
  default:
    console.log("Unknown day");
}
```

Output:

```
Working day
```

2. `switch` Mental Model

`switch` -কে এভাবে ভাবতে পারো:



অর্থাতঃ:

একটি *value* নিয়ে বিভিন্ন `case`-এর সাথে *match* করা হয়।

3. Basic Syntax

```
switch (expression) {  
    case value1:  
        // code  
        break;  
  
    case value2:  
        // code  
        break;  
  
    default:  
        // code  
}
```

এখানে:

- `switch` → যে value check করবে
- `case` → সম্ভাব্য value
- `break` → switch থেকে বের হয়ে যাবে
- `default` → কোনো case match না হলে

4. Simple Example

```
const fruit = "apple";

switch (fruit) {
  case "apple":
    console.log("Apple");
    break;

  case "banana":
    console.log("Banana");
    break;

  case "orange":
    console.log("Orange");
    break;
}
```

Output:

```
Apple
```

5. **case**

case দিয়ে possible value define করা হয়।

```
switch (role) {
  case "admin":
    console.log("Admin");
    break;

  case "user":
    console.log("User");
    break;
}
```

যদি:

```
role === "admin"
```

তাহলে প্রথম case execute হবে।

6. **break**

break অত্যন্ত important!

```
const color = "red";

switch (color) {
  case "red":
    console.log("Red");
    break;

  case "blue":
    console.log("Blue");
    break;
}
```

Output:

```
Red
```

break execution-কে switch-এর বাইরে নিয়ে যায়।

7. **break** না দিলে কী হয়?

এখানেই আসে **fall-through**।

```
const number = 1;

switch (number) {
  case 1:
    console.log("One");

  case 2:
    console.log("Two");

  case 3:
    console.log("Three");
}
```

Output:

```
One
  Two
  Three
```

কেন?

`case 1` match করেছে।

কিন্তু `break` নেই।

তাই পরের cases-গুলোও execute হতে থাকে।

এটাকে বলে:

Fall-through

8. Fall-through

Fall-through মানে:

একটি case match করার পর `break` না থাকলে নিচের case-এর code-ও execute হতে থাকে।

Example:

```
const value = 2;

switch (value) {
  case 1:
    console.log("A");
    break;

  case 2:
    console.log("B");

  case 3:
    console.log("C");
    break;
}
```

Output:

```
B
C
```

কারণ `case 2` match করার পরে break ছিল না।

9. Intentional Fall-through

Fall-through সবসময় ভুল নয়।

কখনো multiple cases-এর জন্য একই code execute করতে চাইলে ইচ্ছাকৃতভাবে break বাদ দেওয়া যায়।

Example:

```
const day = "Saturday";

switch (day) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;

  default:
    console.log("Weekday");
}
```

এখানে Saturday এবং Sunday একই code share করছে।

10. Multiple Cases → Same Result

এটা `switch`-এর খুব useful pattern।

```
const day = "Sunday";

switch (day) {
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;

  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Weekday");
    break;

  default:
    console.log("Invalid day");
}
```

11. default

কোনো `case` match না করলে `default` execute হয়।

```
const role = "guest";

switch (role) {
  case "admin":
    console.log("Admin");
    break;

  case "user":
    console.log("User");
    break;

  default:
    console.log("Unknown role");
}
```

Output:

```
Unknown role
```

12. default কি বাধ্যতামূলক?

না।

এটা optional।

```
switch (role) {
  case "admin":
    console.log("Admin");
    break;

  case "user":
    console.log("User");
    break;
}
```

কোনো `case` match না করলে কিছু হবে না।

তবে unexpected value handle করতে `default` রাখা ভালো।

13. `default` কোথায় থাকে?

সাধারণত শেষে:

```
switch (value) {  
  case "a":  
    // ...  
    break;  
  
  case "b":  
    // ...  
    break;  
  
  default:  
    // ...  
}
```

Technically `default` অন্য জায়গাতেও থাকতে পারে, কিন্তু readability-এর জন্য শেষে রাখাই best practice।

14. `switch` কি Strict Comparison করে?

হ্যাঁ।

`switch` case matching-এ **strict equality** (`===`) semantics ব্যবহার করে।

Example:

```
const value = 10;

switch (value) {
  case "10":
    console.log("String");
    break;

  case 10:
    console.log("Number");
    break;

  default:
    console.log("Other");
}
```

Output:

Number

কারণ:

```
10 === "10"
  ↓
false
```

কিন্তু:

```
10 === 10
  ↓
true
```

15. **switch** with Strings

```
const role = "admin";

switch (role) {
  case "admin":
    console.log("Full access");
    break;

  case "editor":
    console.log("Edit access");
    break;

  case "viewer":
    console.log("Read-only access");
    break;

  default:
    console.log("No access");
}
```

16. `switch` with Numbers

```
const statusCode = 200;

switch (statusCode) {
  case 200:
    console.log("Success");
    break;

  case 404:
    console.log("Not found");
    break;

  case 500:
    console.log("Server error");
    break;

  default:
    console.log("Unknown status");
}
```

17. `switch` with Boolean

Technically করা যায়:

```
const isLoggedIn = true;

switch (isLoggedIn) {
  case true:
    console.log("Logged in");
    break;

  case false:
    console.log("Logged out");
    break;
}
```

তবে simple Boolean-এর জন্য `if / else` সাধারণত বেশি readable:

```
if (isLoggedIn) {  
  console.log("Logged in");  
} else {  
  console.log("Logged out");  
}
```

18. **switch** Expression

switch শুধু variable-এর সাথে ব্যবহার করতে হবে এমন নয়।

Expression ব্যবহার করা যায়:

```
const age = 20;  
  
switch (age >= 18) {  
  case true:  
    console.log("Adult");  
    break;  
  
  case false:  
    console.log("Minor");  
    break;  
}
```

তবে এটাও **if / else** দিয়ে অনেক বেশি natural:

```
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

19. **switch** vs **if / else**

switch

Fixed values match করার জন্য ভালো:

```
switch (role) {  
    case "admin":  
        // ...  
        break;  
  
    case "editor":  
        // ...  
        break;  
  
    case "viewer":  
        // ...  
        break;  
}
```

if / else

Ranges বা complex conditions-এর জন্য ভালো:

```
if (score >= 80) {  
    // A+  
} else if (score >= 70) {  
    // A  
}
```

20. কখন **switch** ব্যবহার করবে?

switch ভালো যখন:

- একটি variable-এর exact values compare করবে
- অনেকগুলো fixed cases আছে
- একই variable-এর different possible states আছে
- menu/action type handle করতে হবে
- status/type/category অনুযায়ী action নিতে হবে

Example:

```
role:
  admin
  editor
  viewer
  guest
```

এক্ষেত্রে `switch` readable হতে পারে।

21. কখন `if / else` ব্যবহার করবে?

`if / else` ভালো যখন:

- ranges check করতে হবে
- multiple variables involve করছে
- complex logical conditions আছে
- `>`, `<`, `>=`, `<=` ইত্যাদি ব্যবহার হচ্ছে
- conditions একে অপরের ওপর depend করছে

Example:

```
if (age >= 18 && isLoggedIn && !isBanned) {
  console.log("Access granted");
}
```

এখানে `switch` natural না।

22. Ternary Operator কী?

Ternary operator হলো JavaScript-এর একটি **conditional expression**।

Syntax:

```
condition ? valueIfTrue : valueIfFalse
```

Example:

```
const age = 20;

const result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

```
Adult
```

23. Ternary-এর Mental Model

```
condition
  ↓
true?
 /   \
Yes   No
  ↓   ↓
value1 value2
```

Example:

```
age >= 18 ? "Adult" : "Minor"
```

মানে:

```
age >= 18?
  ↓
Yes → "Adult"
No  → "Minor"
```

24. Ternary Syntax Breakdown

```
condition ? trueValue : falseValue
```

এখানে:

```
condition
  ↓
  ?
  ↓
true value
  ↓
  :
  ↓
false value
```

Example:

```
const message = isLoggedIn
  ? "Welcome"
  : "Please login";
```

25. Ternary is an Expression

এটা খুব important।

Ternary value produce করে।

তাই variable-এ assign করা যায়:

```
const message = age >= 18 ? "Adult" : "Minor";
```

Function থেকে return করা যায়:

```
function getStatus(isLoggedIn) {
  return isLoggedIn ? "Online" : "Offline";
}
```

26. Ternary with Function Return

```
function getDiscount(isMember) {
  return isMember ? 20 : 0;
}
```

যদি:

```
getDiscount(true);
```

তাহলে:

```
20
```

আর:

```
getDiscount(false);
```

তাহলে:

```
0
```

27. Ternary with Template Literals

```
const age = 25;

const message = `You are ${
  age >= 18 ? "an adult" : "a minor"
}.`;
```

Output:

```
You are an adult.
```

28. Ternary with Function Call

```
const isLoggedIn = true;

const message = isLoggedIn
  ? getUsername()
  : "Guest";
```

এখানে condition অনুযায়ী function call হবে।

29. Ternary in React

React-এ ternary অত্যন্ত common।

```
function App({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? (  
        <h1>Welcome</h1>  
      ) : (  
        <h1>Please Login</h1>  
      )}  
    </div>  
  );  
}
```

এখানে:

```
isLoggedIn?  
↓  
Yes → Welcome  
No  → Please Login
```

30. React Loading Example

```
function UserList({ loading }) {  
  return (  
    <div>  
      {loading ? (  
        <p>Loading...</p>  
      ) : (  
        <p>Users loaded</p>  
      )}  
    </div>  
  );  
}
```

31. React Error Example

```
function UserData({ error }) {  
  return (  
    <div>  
      {error ? (  
        <p>Something went wrong</p>  
      ) : (  
        <p>Data loaded successfully</p>  
      )}  
    </div>  
  );  
}
```

32. React Login Button Example

```
function Header({ isLoggedIn }) {  
  return (  
    <button>  
      {isLoggedIn ? "Logout" : "Login"}  
    </button>  
  );  
}
```

এটা ternary-এর perfect use case।

কারণ:

দুইটা possible value-এর মধ্যে একটা select করতে হচ্ছে।

33. Ternary vs `if / else`

`if / else`

```
let message;  
  
if (isLoggedIn) {  
  message = "Welcome";  
} else {  
  message = "Please login";  
}
```

Ternary

```
const message = isLoggedIn  
  ? "Welcome"  
  : "Please login";
```

দুটোই valid।

Ternary ছোট expression-এর জন্য cleaner হতে পারে।

34. কখন Ternary ব্যবহার করবে?

Ternary ভালো যখন:

- simple condition
- দুইটি possible result
- variable assign করতে হবে
- return value select করতে হবে
- JSX-এ ছোট conditional UI দরকার

Example:

```
const status = online ? "Online" : "Offline";
```

35. কখন Ternary ব্যবহার করবে না?

যদি logic অনেক complex হয়:

```
const result = condition1
  ? condition2
  ? condition3
  ? "A"
  : "B"
  : "C"
  : "D";
```

এটা technically valid হলেও readability খারাপ।

এক্ষেত্রে `if / else` ব্যবহার করা ভালো।

36. Nested Ternary

Ternary-এর ভিতরে আবার ternary ব্যবহার করা যায়।

```
const score = 85;

const grade =
  score >= 80
    ? "A+"
    : score >= 70
      ? "A"
      : "Fail";
```

এখানে:

```
score >= 80?
  ↓
Yes → A+
No
  ↓
score >= 70?
  ↓
Yes → A
No  → Fail
```

37. Nested Ternary কেন Avoid করা হয়?

কারণ পড়তে difficult হয়ে যায়।

Example:

```
const status =
  isLoading
    ? "Loading"
    : hasError
      ? "Error"
      : hasData
        ? "Success"
        : "Empty";
```

এটা বোঝা সম্ভব, কিন্তু বড় হলে maintain করা কঠিন।

Complex logic হলে:

```
if (isLoading) {  
    return "Loading";  
}  
  
if (hasError) {  
    return "Error";  
}  
  
if (hasData) {  
    return "Success";  
}  
  
return "Empty";
```

এটা বেশি readable।

38. Ternary + Assignment

Common pattern:

```
const role = isAdmin ? "Admin" : "User";
```

এখানে `role` একটি value পাবে।

39. Ternary + Default Value

```
const name = username ? username : "Guest";
```

এটা কাজ করবে।

তবে যদি উদ্দেশ্য হয় শুধু nullish value handle করা, পরে আমরা `??` শিখব:

```
const name = username ?? "Guest";
```

`??` বিস্তারিত **Doc 8.3**-তে থাকবে।

40. Ternary + Boolean

```
const status = isOnline ? "Online" : "Offline";
```

Simple এবং readable।

41. Ternary + Number

```
const discount = isMember ? 20 : 0;
```

42. Ternary + Function Return

```
function getAccess(user) {  
  return user.isAdmin ? "Full access" : "Limited access";  
}
```

Simple two-way result হলে সুন্দর।

43. **switch** vs Ternary

এগুলো একই ধরনের problem solve করে না।

switch

অনেক fixed cases:

```
switch (role) {  
  case "admin":  
    // ...  
    break;  
  
  case "editor":  
    // ...  
    break;  
  
  case "viewer":  
    // ...  
    break;  
}
```

Ternary

দুইটা result:

```
const label = isAdmin ? "Admin" : "User";
```

44. if / else vs Ternary vs switch

Situation	Best choice
One simple condition	if
Two-way value selection	Ternary
Multiple fixed values	switch
Range comparison	if / else if
Complex logic	if / else
React small conditional UI	Ternary
Many exact states	switch

45. Real-world Example — Order Status

```
const status = "shipped";

switch (status) {
  case "pending":
    console.log("Order is pending");
    break;

  case "processing":
    console.log("Order is being processed");
    break;

  case "shipped":
    console.log("Order has been shipped");
    break;

  case "delivered":
    console.log("Order delivered");
    break;

  case "cancelled":
    console.log("Order cancelled");
    break;

  default:
    console.log("Unknown order status");
}
```

এখানে `switch` খুব natural।

46. Real-world Example — Theme

Simple দুইটা option:

```
const theme = isDarkMode ? "dark" : "light";
```

Ternary যথেষ্ট।

47. Real-world Example — Permission

```
const message = isAdmin
  ? "You have full access"
  : "You have limited access";
```

48. Real-world Example — Payment Status

```
const status = "failed";

switch (status) {
  case "pending":
    console.log("Payment pending");
    break;

  case "success":
    console.log("Payment successful");
    break;

  case "failed":
    console.log("Payment failed");
    break;

  default:
    console.log("Unknown payment status");
}
```

49. Real-world Example — HTTP Status

```
const statusCode = 404;

switch (statusCode) {
  case 200:
    console.log("Success");
    break;

  case 201:
    console.log("Created");
    break;

  case 400:
    console.log("Bad request");
    break;

  case 401:
    console.log("Unauthorized");
    break;

  case 404:
    console.log("Not found");
    break;

  case 500:
    console.log("Server error");
    break;

  default:
    console.log("Unknown status");
}
```

50. Real-world React Example — Status Badge

```
function Status({ status }) {  
  return (  
    <span>  
      {status === "active"  
        ? "Active"  
        : "Inactive"}  
    </span>  
  );  
}
```

Simple two-way UI, তাই ternary appropriate।

51. React Multiple Conditions

যদি multiple states থাকে:

```
function Status({ status }) {  
  if (status === "loading") {  
    return <p>Loading...</p>;  
  }  
  
  if (status === "error") {  
    return <p>Error occurred</p>;  
  }  
  
  if (status === "success") {  
    return <p>Success</p>;  
  }  
  
  return <p>Unknown state</p>;  
}
```

এখানে `if` অনেক সময় nested ternary-এর চেয়ে readable।

52. React: Ternary vs `if`

Ternary ভালো:

```
return (  
  <button>  
    {isLoggedIn ? "Logout" : "Login"}  
  </button>  
);
```

`if` ভালো:

```
if (loading) {  
  return <Loading />;  
}  
  
if (error) {  
  return <Error />;  
}  
  
return <Content />;
```

Rule

JSX-এর মধ্যে ছোট two-way decision → ternary
Component-level complex branching → `if`

53. Common Mistake — `break` Forget করা

✗:

```
switch (role) {  
  case "admin":  
    console.log("Admin");  
  
  case "user":  
    console.log("User");  
}
```

এখানে fall-through হবে।

সাধারণত:

```
switch (role) {  
  case "admin":  
    console.log("Admin");  
    break;  
  
  case "user":  
    console.log("User");  
    break;  
}
```

54. Common Mistake — **break** সবসময় ভুলভাবে ব্যবহার করা

Multiple cases-এর same output চাইলে:

```
switch (day) {  
  case "Saturday":  
  case "Sunday":  
    console.log("Weekend");  
    break;  
}
```

এখানে প্রথম case-এ break না থাকাটা intentional।

55. Common Mistake — Type Difference

```
const value = 1;

switch (value) {
  case "1":
    console.log("String");
    break;

  case 1:
    console.log("Number");
    break;
}
```

Output:

```
Number
```

কারণ strict matching!

56. Common Mistake — Overusing Ternary

✗:

```
const result = a
  ? b
  ? c
  ? "X"
  : "Y"
  : "Z"
  : "W";
```

এটা code review-এ readability issue তৈরি করতে পারে।

Better:

```
if (a) {  
  if (b) {  
    return "X";  
  }  
  
  return "Z";  
}  
  
return "W";
```

57. Common Mistake — Ternary for Side Effects

এভাবে লেখা technically possible:

```
isLoggedIn ? console.log("Yes") : console.log("No");
```

কিন্তু এখানে `if/else` বেশি readable:

```
if (isLoggedIn) {  
  console.log("Yes");  
} else {  
  console.log("No");  
}
```

Ternary-এর মূল উদ্দেশ্য value select করা।

58. Common Mistake — Huge Ternary

যদি ternary কয়েক line জুড়ে complicated হয়ে যায়:

থামো এবং `if / else` consider করো।

Readable code > clever code.

59. Output Practice — switch

Question 1

```
const value = 2;

switch (value) {
  case 1:
    console.log("One");
    break;

  case 2:
    console.log("Two");
    break;

  default:
    console.log("Other");
}
```

Output कौ?

Question 2

```
const value = 2;

switch (value) {
  case 1:
    console.log("One");

  case 2:
    console.log("Two");

  case 3:
    console.log("Three");
    break;

  default:
    console.log("Other");
}
```

Output कै?

Question 3

```
const value = "10";

switch (value) {
  case 10:
    console.log("Number");
    break;

  case "10":
    console.log("String");
    break;
}
```

60. Output Practice — Ternary

Question 4

```
const age = 20;

const result = age >= 18
  ? "Adult"
  : "Minor";

console.log(result);
```

Question 5

```
const isLoggedIn = false;

const message = isLoggedIn
  ? "Welcome"
  : "Please login";

console.log(message);
```

Question 6

```
const score = 75;

const grade =
  score >= 80
    ? "A+"
    : score >= 70
      ? "A"
      : "Fail";

console.log(grade);
```

61. Coding Challenge 1 — Day Name

Function:

```
getDayName(dayNumber)
```

Rules:

```
1 → Sunday
2 → Monday
3 → Tuesday
4 → Wednesday
5 → Thursday
6 → Friday
7 → Saturday
```

Invalid number:

```
"Invalid day"
```

`switch` দিয়ে implement করো।

62. Coding Challenge 2 — Order Status

Function:

```
getOrderMessage(status)
```

Rules:

```
pending    → "Order is pending"
processing → "Order is processing"
shipped    → "Order shipped"
delivered  → "Order delivered"
cancelled  → "Order cancelled"
```

Unknown status:

```
"Unknown status"
```

`switch` ব্যবহার করো।

63. Coding Challenge 3 — Login Label

একটি variable:

```
const isLoggedIn = true;
```

Ternary ব্যবহার করে:

```
true  → "Logout"
false → "Login"
```

return/assign করো।

Expected:

```
const label = isLoggedIn ? "Logout" : "Login";
```

64. Coding Challenge 4 — Discount

Function:

```
getDiscount(isMember)
```

Rules:

```
true → 20  
false → 0
```

Ternary ব্যবহার করো।

65. Coding Challenge 5 — Status Message

Input:

```
const status = "loading";
```

Rules:

```
loading → "Loading..."  
error → "Something went wrong"  
success → "Data loaded"  
empty → "No data"
```

`switch` দিয়ে implement করো।

66. Interview Questions

Q1. `switch` কী?

একটি expression-এর value-এর সাথে multiple fixed `case` value compare করে appropriate code execute করার control-flow statement।

Q2. `switch` case কীভাবে compare করে?

Strict equality semantics ব্যবহার করে।

অর্থাৎ:

```
10
```

এবং:

```
"10"
```

একই case নয়।

Q3. `break` কেন ব্যবহার করা হয়?

Matching case execute হওয়ার পর switch-এর execution বন্ধ করে বের হওয়ার জন্য।

Q4. `break` না দিলে কী হয়?

Fall-through হতে পারে এবং পরবর্তী cases-এর code execute হতে পারে।

Q5. Fall-through কি সবসময় bug?

না। Multiple cases-এর same logic intentionally share করতে এটি ব্যবহার করা যায়।

Q6. `default` কী?

কোনো case match না করলে যে block execute হয়।

Q7. Ternary operator কী?

একটি condition-এর ভিত্তিতে দুইটি expression/value-এর মধ্যে একটি নির্বাচন করার concise syntax।

```
condition ? trueValue : falseValue
```

Q8. Ternary কি statement নাকি expression?

Ternary হলো **expression** এবং একটি value produce করে।

Q9. কখন ternary ব্যবহার করা ভালো?

Simple two-way value selection-এর জন্য।

Q10. Nested ternary কেন avoid করা হয়?

Logic বেশি complex হয়ে গেলে readability এবং maintainability কমে যায়।

Q11. `switch` আর `if/else` কখন কোনটা?

- Fixed exact values → `switch`
- Ranges/complex conditions → `if/else`

Q12. React-এ ternary কোথায় useful?

JSX-এর মধ্যে simple conditional rendering-এর জন্য।

Example:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

67. 🧠 Final Mental Model

if / else

Condition

↓

true? — Yes → Action A

|

No

↓

Action B

switch

Value

↓

case 1?

↓

case 2?

↓

case 3?

↓

default

Ternary

Condition

↓

true?

/

\

A

B

68. 🔥 Golden Rules

1. `switch` fixed/exact values compare করার জন্য ভালো।
2. `case` দিয়ে possible values define করা হয়।
3. সাধারণ `case` block-এর শেষে `break` ব্যবহার করো।

- 4. `break` না থাকলে fall-through হতে পারে।
- 5. Multiple cases-এর same output হলে intentional fall-through ব্যবহার করা যায়।
- 6. `default` unmatched values handle করে।
- 7. `switch` strict equality semantics ব্যবহার করে।
- 8. Ternary syntax:

```
condition ? trueValue : falseValue
```

- 9. Ternary একটি expression এবং value return করে।
- 10. Simple two-way decision → ternary।
- 11. Complex decision → `if / else`।
- 12. Multiple fixed states → `switch`।
- 13. Range condition → `if / else if`।
- 14. React JSX-এর ছোট conditional rendering-এর জন্য ternary খুব useful।
- 15. Nested ternary অতিরিক্ত হলে `if/else` ব্যবহার করো।
- 16. Code ছোট করার চেয়ে **readability বেশি important**।



Quick Revision

Concept	Best Use
<code>if</code>	Single condition
<code>if / else</code>	Two-way logic
<code>else if</code>	Multiple conditions
<code>switch</code>	Multiple exact values
<code>case</code>	Specific switch value
<code>break</code>	Stop fall-through
<code>default</code>	No case matched
Fall-through	Multiple cases share logic
Ternary	Simple two-way value
Nested ternary	Use sparingly
JSX ternary	Conditional UI



Mastery Checklist

- ☐ `switch` syntax জানি
- ☐ `case` বুঝি
- ☐ `break` বুঝি
- ☐ Fall-through বুঝি
- ☐ Intentional fall-through বুঝি
- ☐ `default` বুঝি
- ☐ `switch` strict matching বুঝি
- ☐ String/number cases handle করতে পারি
- ☐ Multiple cases combine করতে পারি
- ☐ `switch` vs `if/else` বুঝি
- ☐ Ternary syntax জানি
- ☐ Ternary expression বুঝি
- ☐ Ternary দিয়ে value assign করতে পারি
- ☐ Ternary দিয়ে function return করতে পারি
- ☐ React JSX-এ ternary ব্যবহার করতে পারি
- ☐ Nested ternary-এর সমস্যা বুঝি
- ☐ কখন `switch`, কখন `if`, কখন ternary ব্যবহার করতে হবে জানি



Next Doc



Doc 8.3 — Nullish Coalescing `??` & Optional Chaining `?.`

এখানে আমরা modern JavaScript-এর খুব important দুইটা operator শিখব:

```
user?.address?.city ?? "Unknown"
```

এর মধ্যে:

```
?. → safely access data  
?? → null/undefined হলে fallback value
```

এগুলো **React + API + Next.js development-এ প্রতিদিনই** দেখতে পাবো।